Multilayered networks with sklearn

Coding: Flow and results

Instructions

- Sort the code fragments.
  - Note: Several fragments span more than one slide.

- For each fragment
  - Describe the fragment
  - Describe or show its results

Loading data

We are going to load the data set using the *read_csv(...)* function from the **pandas** library. Note that, if the data set file is not in the same location as the notebook, we need to indicate its path. If you are using Google Colab, this platform has its own way for uploading and accessing files.

```
datos=pd.read_csv("breast-model.csv",header=None)
datos.head()
```

[2]:

	0	1	2	3	4	5	6	7	8	9
0	5	1	1	1	2	1	3	1	1	2
1	5	4	4	5	7	10	3	2	1	2
2	4	1	1	3	2	1	3	1	1	2
3	1	1	1	1	2	10	3	1	1	2
4	2	1	2	1	2	1	3	1	1	2

```
print(datos.shape) #Show the dimensions of the data set
```

```
(694, 10)
```

```
X=datos.to_numpy()[:,0:9] #Features
y=datos.to_numpy()[:,9] #Labels (classes)
```

B

Preparing the data

Besides dividing into features and labels, we are also going to split the data set into training and test sets. This is done with the *train_test_split(...)* method. We therefore will have the features for training (X_train), the labels for training (y_train), the features for testing (X_test), and the labels for testing (y_test).

We are also going to scale the data.

```
#Training set and test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.3, random_state=42)

print(X_train.shape) #Size of the training set
print(X_test.shape) #Size of the test set
```

```
(485, 9)
(209, 9)
```

```
#Standardizing the training and test sets
X_train = StandardScaler().fit_transform(X_train)
X_test = StandardScaler().fit_transform(X_test)
```

C

```
X_uso=datos_uso.to_numpy()[:,0:9] #Features
y_uso=datos_uso.to_numpy()[:,9] #Labels
```

```
X_uso
```

7]:
```
array([[ 3,  1,  1,  1,  2,  2,  3,  1,  1],
       [ 6,  8,  8,  1,  3,  4,  3,  7,  1],
       [ 8, 10, 10,  8,  7, 10,  9,  7,  1],
       [ 8,  7,  4,  4,  5,  3,  5, 10,  1],
       [ 6,  2,  3,  1,  2,  1,  1,  1,  1]], dtype=int64)
```

```
#Normally, these labels are not available (this is only to illustrate network use)
y_uso
```

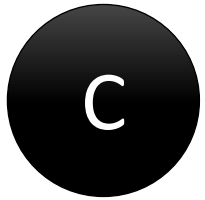
8]:
```
array([2, 2, 4, 4, 2], dtype=int64)
```

```
X_uso = StandardScaler().fit_transform(X_uso)
```

```
y_pred=clf.predict(X_uso)
y_pred
```

0]:
```
array([2, 4, 4, 2, 2], dtype=int64)
```

As we an see, the neural network classified 3/5 instances correctly (0.6 accuracy).



Use

For this demo, we have previously taken some instances from the *Breast Cancer Wisconsin* data set. These instances **were not** used neither for training nor for test, and we use the trained network for obtaining the corresponding labels. Conventionally, these instances would come from an application or from an operations data repository, and thus we do not know their labels.

```
#Generally, we would first save the model and then we would deploy it.
#Here, we are only seeing how it can be used
datos_uso=pd.read_csv("breast-usage.csv",header=None)
datos_uso
```

15]:

	0	1	2	3	4	5	6	7	8	9
0	3	1	1	1	2	2	3	1	1	2
1	6	8	8	1	3	4	3	7	1	2
2	8	10	10	8	7	10	9	7	1	4
3	8	7	4	4	5	3	5	10	1	4
4	6	2	3	1	2	1	1	1	1	2

```
X_uso=datos_uso.to_numpy()[:,0:9] #Features
y_uso=datos_uso.to_numpy()[:,9] #Labels
```

```
X_uso
```

17]:

```
array([[ 3,  1,  1,  1,  2,  2,  3,  1,  1],
       [ 6,  8,  8,  1,  3,  4,  3,  7,  1],
       [ 8, 10, 10,  8,  7, 10,  9,  7,  1],
       [ 8,  7,  4,  4,  5,  3,  5, 10,  1],
       [ 6,  2,  3,  1,  2,  1,  1,  1,  1]], dtype=int64)
```

B

```
print("=== STANDARDIZATION ===")
print(" +++ Training +++")
print(X_train[:5])
```

```
=== STANDARDIZATION ===
 +++ Training +++
[[ 1.32939178  2.26488639  0.63000933  0.08740615  2.17061909  0.19852479
   0.27965722  2.45054764  0.90946449]
 [-0.12521626 -0.6897547  -0.72499965 -0.62506416 -0.53567978 -0.65104453
  -0.13896547 -0.59432581 -0.33304333]
 [ 1.32939178  1.60829948  1.64626606 -0.62506416 -0.53567978 -0.9342343
   1.11690261  2.45054764 -0.33304333]
 [-0.48886827 -0.36146125 -0.3862474   0.08740615 -0.53567978 -0.08466498
  -0.13896547 -0.59432581 -0.33304333]
 [-1.21617229 -0.6897547  -0.72499965 -0.62506416 -0.98672959 -0.65104453
  -0.13896547 -0.59432581 -0.33304333]]
```

```
print(" +++ Test +++")
print(X_test[:5])
```

```
 +++ Test +++
[[-0.86074001 -0.7127752  -0.77645105 -0.65059447 -0.57479539 -0.73415688
  -1.05783862 -0.64220756 -0.37420267]
 [-0.52259215 -0.38173072 -0.77645105 -0.31235008 -0.57479539 -0.73415688
  -0.26398486 -0.64220756 -0.37420267]
 [ 1.84444289  0.61140273  1.57054871  2.39360502 -0.1214662   1.5985274
   0.5298689  -0.64220756  0.65485467]
 [-1.19888788 -0.7127752  -0.77645105 -0.65059447 -0.57479539 -0.73415688
  -0.26398486 -0.64220756 -0.37420267]
 [-0.52259215 -0.7127752  -0.77645105 -0.65059447 -0.57479539 -0.73415688
  -0.26398486 -0.64220756 -0.37420267]]
```

Test

In this case, we evaluate the model's quality using the *score(...)* method, which calculates the proportion of instances that were correctly classified. However, sklearn offers an important variety of methods for knowing if our neural network generalized well.

```
#We evaluate the model using the test set.
accuracy=round(clf.score(X_test, y_test),3)
print("***Accuracy: ",accuracy,"***")
```

```
***Accuracy:  0.981 ***
```

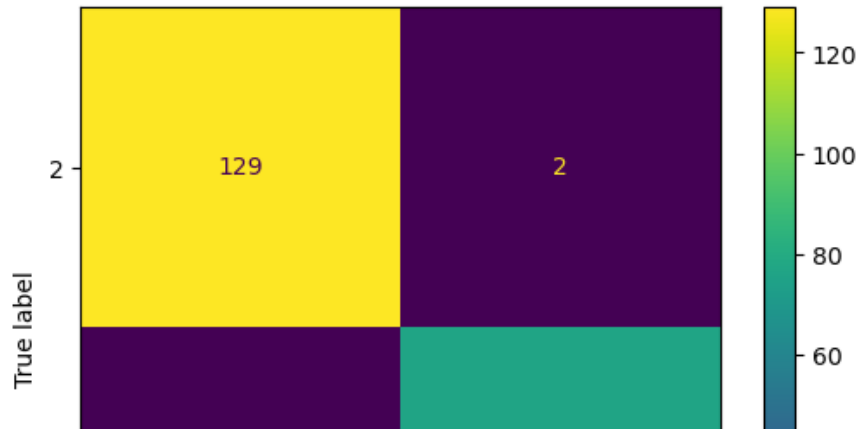
```
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred, target_names=["Class 2","Clases 4"]))
```

```
              precision    recall  f1-score   support

     Class 2       0.98      0.98      0.98       131
    Clases 4       0.97      0.97      0.97        78

    accuracy                           0.98       209
   macro avg       0.98      0.98      0.98       209
weighted avg       0.98      0.98      0.98       209
```

```
#Confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=clf.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=clf.classes_)
disp.plot(values_format = '')
plt.show()
```




```
print("=== ORIGINAL (first lines) ===")
print(X[:5]) #Shows the first five lines of the data set
```

```
=== ORIGINAL (first lines) ===
[[ 5  1  1  1  2  1  3  1  1]
 [ 5  4  4  5  7 10  3  2  1]
 [ 4  1  1  3  2  1  3  1  1]
 [ 1  1  1  1  2 10  3  1  1]
 [ 2  1  2  1  2  1  3  1  1]]
```

```
print(y[:5]) #Shows the first five labels
```

```
[2 2 2 2 2]
```

In this case, we take the first nine columns as features and the tenth column as the label. Let us consider that, for other data sets, we need to first check which columns respectively correspond to features and labels. *Tip:* If the data set is small, we can modify it using applications such as Excel, such that the label is always the last column. Otherwise, we could modify it using pandas.

Multilayer networks

In this demo, we are going to train, test, and use a multilayer neural network. We are also going to see how to load data using pandas and how to pre-process this data using scikit-learn. This library will also serve to generate a learning model.

:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier

import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Training

We use *MLPClassifier* from the **scikit-learn** (sklearn) library with *fit()* in order to train the model.

```
clf=MLPClassifier(random_state=42,max_iter=1000) #Tuning the model's hyperparameters
clf.fit(X_train, y_train) #Training
```

1]: MLPClassifier(max_iter=1000, random_state=42)